Solidity – Annotated Programs

Read it, write it, remember it

Part 1 – The 4 steps to write any contract

Type the header first. It is always the same three lines, then fill the contract in this order:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Name {
    // 1 DATA   – what must be remembered?
    // 2 SETUP  – constructor (only if needed)
    // 3 DO     – functions that CHANGE data  (RULE with require first, then
the change)
    // 4 SHOW   – functions that only READ    (view / pure)
}
```

1. **1 DATA** – variables, `mapping`, `struct`, arrays. Ask: what does the question want me to remember?
2. **2 SETUP** – `constructor(...)` runs once when deployed (set the owner or starting values). Skip it if not needed.
3. **3 DO** – a function that changes data. If the question has a rule ("only once", "enough balance", "owner only"), write `require(rule, "message");` on the first line of the function, then do the change.
4. **4 SHOW** – a function that only reads: `view` if it reads stored data, `pure` if it only uses its inputs (pure maths). Both are free to call.

Question words → code

- "number / count / balance" → `uint x;`
- "name / text" → `string x;`
- "yes or no / already done" → `bool x;`
- "for each user" → `mapping(address => uint)`; "for each id" → `mapping(uint => ...)`
- "student / candidate / account with several fields" → `struct`, then a list `Name[] list;` or a mapping of it

- "only the owner" → `require(msg.sender == owner, "msg");`
- "only once / not twice" → `require(!done[msg.sender], "msg");`
- "enough balance" → `require(balance[msg.sender] >= amt, "msg");`
- "pay / deposit Ether" → add `payable`, the amount is `msg.value`
- "send Ether to someone" → `payable(to).transfer(amt);`
- "child / extends" → `contract B is A` and the constructor passes values up: `B(...) A(x, y) {}`
- "just calculate" → `pure returns (...)`; "show / get" → `view returns (...)`

Words you will see

- `public` – anyone can call it (a public variable gets a free reader function)
- `view` – only reads stored data, free
- `pure` – uses only its inputs, free
- `payable` – the function can receive Ether
- `memory` – temporary copy; needed for text and list inputs (`string memory name`)
- `msg.sender` – the wallet that called the function
- `msg.value` – the Ether sent with the call (in wei)
- `require(rule, "msg")` – if the rule is false, stop and undo everything
- `mapping(a => b)` – like a dictionary
- `struct` – your own record with named fields
- `event` / `emit` – write a log line the front end can read
- `modifier` – a reusable rule written once and attached to functions
- `constructor` – runs once at deploy
- `is` – inheritance (`contract Car is Vehicle`)
- `address(this).balance` – the Ether held by the contract itself

Part 2 – Practical 4: Basics in Remix

Program 1 – Print a simple message

Stores one text. Anyone can read it (public) and change it.

Remember: public variable = free reader. Setter = one line.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// 1 DATA + 3 DO: this contract has one variable and one function
contract MessageContract {
    // 1 DATA: the stored message. `public` = Solidity gives a free getter
called ms()
    string public ms = "Hello MSc CS Students";
    // 3 DO: changes the data, so no view/pure. Input is text, so `memory`
    function setMessage(string memory nm) public { ms = nm; }
}
```

Program 2 – Addition of two numbers

Takes two numbers, returns the sum. Stores nothing.

Remember: Nothing stored = `pure`. Shape: function(inputs) public pure returns (type).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.6;

contract AdditionContract {
    // 4 SHOW (pure): no data stored, just inputs -> answer. int = number
that can be negative
    function AddNumber(int num01, int num02) public pure returns (int) {
return num01 + num02; }
}
```

Program 3 – Arithmetic operations on two numbers

Four pure functions: add, subtract, multiply, divide.

Remember: Copy the same one-line function 4 times and change the operator.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract AdditionContract {
    // 4 SHOW (pure): n1 + n2
    function AddNumbers(int n1, int n2) public pure returns (int) { return n1
+ n2; }
    // same shape, operator is -
    function SubNumbers(int n1, int n2) public pure returns (int) { return n1
- n2; }
    // same shape, operator is *
    function MultiplyNumbers(int n1, int n2) public pure returns (int) {
return n1 * n2; }
    // same shape, operator is /  (whole-number division)
    function DivideNumbers(int n1, int n2) public pure returns (int) { return
n1 / n2; }
}
```

Program 4 – Palindrome number

Reverse the number digit by digit and compare with the original.

Remember: Same trick as Python: last digit = n % 10, drop it = n / 10.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.6;

contract Palindrome {
    // 4 SHOW (pure): returns true/false (bool)
    function IsPalindrome(int num) public pure returns (bool) {
        // one digit is always a palindrome
        if (num < 10) return true;
        // keep a copy of the original, because the loop destroys num
        int temporary = num;
        // the answer being built (starts at 0)
        int reversed;
        // loop: take the last digit (num % 10), push it onto reversed, then
cut it off (num /= 10)
        for (; num > 0; num /= 10) reversed = reversed * 10 + num % 10;
        // palindrome if the reversed number equals the original
        return temporary == reversed;
    }
}
```

Program 5 – Array of student names, count and list

A list of names. One function adds a name, one gives the count.

Remember: Array: .push(x) to add, .length for the count.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.6;

contract StudentCount {
    // 1 DATA: a list of texts. `public` also lets you read item number i
    string[] public StudentArray;
    // 3 DO: .push adds to the end of the list
    function addStudent(string memory name) public { StudentArray.push(name);
}
    // 4 SHOW (view): reads the data, so `view`. .length = how many
    function getCount() public view returns (uint) { return
StudentArray.length; }
}
```

Program 6 – Mean, median and mode of an array

Store a number list, then calculate from it.

Remember: Mean = loop and add, divide by length. Median = the middle item. Mode is simplified here.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Stats {
    // 1 DATA: the list of numbers
    uint[] public a;
    // 3 DO: replace the list with the one given (input arrays are `memory`)
    function set(uint[] memory x) public { a = x; }
    // 4 SHOW (view): add everything, then divide by how many
    function mean() public view returns (uint) {
        // running total (starts at 0 by default)
        uint s;
        // go through every item: s += a[i]
        for (uint i; i < a.length; i++) s += a[i];
        return s / a.length;
    }
    // middle item = index length/2 (list must be entered sorted)
    function median() public view returns (uint) { return a[a.length / 2]; }
// middle element (array entered sorted)
    // simplified: just returns the first item
    function mode() public view returns (uint) { return a[0]; }
// simplified, as in the original
}
```

Part 3 – Practical 5: Structures

Practical 5 – Q1 A: Structure for one student

One student's record stored in a struct; get the name and a grade from the marks.

Remember: struct = your own record. Build it with student(id, name, ...) in the field order.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract studInfo {
```

```
    // 1 DATA (a): the fields of one record (a struct is like a Python class
with only fields)
    struct student { int id; string name; string addres; int age; uint[]
marks; }
    // 1 DATA (b): one variable of that record type
    student s;

    // 3 DO: fill the whole record at once, values in the same order as the
fields
    function setValue(int i, string memory n, string memory a, int ag, uint[]
memory m) public {
        s = student(i, n, a, ag, m);
    }
    // 4 SHOW (view): .name reads one field
    function getName() public view returns (string memory) { return s.name; }
    // 4 SHOW (view): average of marks -> grade letter
    function getGrade() public view returns (string memory) {
        // total of marks (starts at 0)
        uint sum;
        // add every mark
        for (uint i; i < s.marks.length; i++) sum += s.marks[i];
        // average = total / how many
        uint avg = sum / s.marks.length;
        // check from the highest grade downwards; first match returns
        if (avg >= 90) return "A+";
        if (avg >= 80) return "A";
        if (avg >= 70) return "B";
        if (avg >= 60) return "C";
        if (avg >= 50) return "D";
        return "F";
    }
}
```

Practical 5 – Q1 B: Structure for multiple students

Same record, but a list of many students.

Remember: struct + array: Student[] list; list.push(Student(...)).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract StudInfo {
```

```
    // 1 DATA (a): the record
    struct Student { int id; string name; string addr; int age; uint[] marks;
}
    // 1 DATA (b): a list of records
    Student[] students;

    // 3 DO: build one record and push it onto the list
    function addStudent(int i, string memory n, string memory a, int ag,
uint[] memory m) public {
        students.push(Student(i, n, a, ag, m));
    }
    // 4 SHOW (view): count = .length
    function totalStudents() public view returns (uint) { return
students.length; }
    // 4 SHOW (view): return the whole list
    function allStudentDetails() public view returns (Student[] memory) {
return students; }

    // 4 SHOW (view): two output lists, one name and one grade per student
    function getAllGrade() public view returns (string[] memory names,
string[] memory grades) {
        // create empty lists of the right size (memory arrays need a size)
        names = new string[](students.length);
        grades = new string[](students.length);
        // outer loop: each student
        for (uint i; i < students.length; i++) {
            names[i] = students[i].name;
            // inner loop adds this student's marks
            uint sum;
            for (uint j; j < students[i].marks.length; j++) sum +=
students[i].marks[j];
            // percentage = total / number of marks
            uint p = sum / students[i].marks.length;
            // pick the grade text from the percentage
            if (p >= 95) grades[i] = "Special Grade";
            else if (p >= 80) grades[i] = "Grade 1";
            else if (p >= 75) grades[i] = "Semi Grade 1";
            else grades[i] = "You are the Potential Man";
        }
    }
}
```

Practical 5 – Q2: Deposit and transactions of a user account

One bank account stored in a struct: create, deposit, withdraw, check balance.

Remember: Withdraw = RULE first (enough money?), then subtract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract BankAccount {
    // 1 DATA (a): account number, name, balance
    struct User { int accNo; string name; uint balance; }
    // 1 DATA (b): the one account
    User u;

    // 3 DO: fill in the account
    function createAccount(int acc, string memory n, uint bal) public { u =
User(acc, n, bal); }
    // 3 DO: balance goes up
    function deposit(uint amount) public { u.balance += amount; }
    // 3 DO with a RULE:
    function withdraw(uint amount) public returns (string memory) {
        // RULE: not enough money -> stop and return the message
        if (u.balance < amount) return "Insufficient Balance";
        // otherwise subtract
        u.balance -= amount;
        return "Transaction Successful";
    }
    // 4 SHOW (view): the balance
    function getBalance() public view returns (uint) { return u.balance; }
}
```

Practical 5 – Q3: Perfect number

A number is perfect if its divisors (below itself) add up to the number.

Remember: Loop to num/2, add every i where num % i == 0.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract PerfectNumber {
    // 4 SHOW (pure): input number -> text answer
    function isPerfect(uint num) public pure returns (string memory) {
        // reject small numbers
        if (num <= 1) return "enter a number greater than 1";
        // start at 1 because 1 divides everything
        uint sum = 1;                                   // 1 divides every
number
        // try every i up to half of num; if it divides evenly, add it
        for (uint i = 2; i <= num / 2; i++) if (num % i == 0) sum += i;
        // perfect if the divisor total equals the number
        if (sum == num) return "It is a perfect number"; return "It is not a
perfect number";
    }
}
```

Part 4 – Practical 6: Mapping

Practical 6 – 1. Voting

Candidates get votes; each address may vote once; find the winner.

Remember: Voting = two mappings (count per candidate, voted yes/no) + two requires.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract voting {
    // 1 DATA: votes per candidate (name -> count)
    mapping(string => uint) public ct;       // votes per candidate
    // 1 DATA: has this wallet voted? (address -> yes/no)
    mapping(address => bool) public voters;  // who has voted
    // 1 DATA: is this name a real candidate?
    mapping(string => bool) isC;             // valid candidate names
    // 1 DATA: list of names (a mapping can't be looped, so keep a list too)
    string[] public cn;                      // candidate list

    // 2 SETUP: runs once at deploy; receives the candidate list
    constructor(string[] memory cdn) {
```

```
        // save the list
        cn = cdn;
        // mark each name as a valid candidate
        for (uint i; i < cdn.length; i++) isC[cdn[i]] = true;
    }
    // 3 DO: cast a vote. RULES first, then changes
    function vote(string memory c) public {
        // RULE 1: not voted before (msg.sender = the caller)
        require(!voters[msg.sender], "You have already voted.");
        // RULE 2: candidate must exist
        require(isC[c], "Candidate does not exist.");
        // add the vote
        ct[c]++;
        // remember that this wallet has voted
        voters[msg.sender] = true;
    }
    // 4 SHOW (view): loop through candidates and keep the highest
    function getWinner() public view returns (string memory w) {
        // t = highest count so far, w = its name
        uint t;
        for (uint i; i < cn.length; i++) if (ct[cn[i]] > t) { t = ct[cn[i]];
w = cn[i]; }
    }
}
```

Practical 6 – 2. Quiz (single-choice, answer typed as text)

Questions with a correct answer; users answer; count correct and wrong.

Remember: Strings can't be compared with == in Solidity: compare their keccak256 hashes.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Quiz {
    // 1 DATA (a): a question and its correct answer
    struct Question { string Qtext; string CA; }   // question text, correct
answer
    // 1 DATA (b): list of questions
    Question[] public q;
    // 1 DATA (c): totals right / wrong
    uint public tc;                                // total correct
```

```
    uint public tw;                                  // total wrong
    // 1 DATA (d): an event = a log line the front end can read
    event AnswerEv(string qu, string uans, bool isc);

    // 3 DO: add a question
    function addQ(string memory qt, string memory cans) public {
q.push(Question(qt, cans)); }
    // 3 DO: answer question number i
    function ansQ(uint i, string memory uans) public {
        // RULE: question number must exist
        require(i < q.length, "Invalid Question Index");
        // compare typed answer with correct answer (hash both, because ==
doesn't work on strings)
        bool ok = keccak256(bytes(uans)) == keccak256(bytes(q[i].CA));   //
strings can't be compared with ==
        // count it right or wrong
        if (ok) tc++; else tw++;
        // write the log line
        emit AnswerEv(q[i].Qtext, uans, ok);
    }
    // 4 SHOW (view): number correct
    function getCAns() public view returns (uint) { return tc; }
}
```

Practical 6 – 3. Quiz (multi-choice, answer picked by option number)

Same quiz, but each question has 4 options and the answer is an option number.

Remember: Numbers compare with ==, so no hashing is needed.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Quiz {
    // 1 DATA (a): text, 4 options, index of the right option
    struct Question { string Qtext; string[4] O; uint8 cO; }   // text, 4
options, correct option index
    // 1 DATA (b): list of questions
    Question[] public q;
    // 1 DATA (c): totals
    uint public tc;
    uint public tw;
    // 1 DATA (d): log line
```

```
    event AnswerRes(string Qtext, string ans, bool isc);

    // 3 DO: add a question
    function addQ(string memory _q, string[4] memory _O, uint8 _cO) public {
        // RULE: correct option must be 0-3
        require(_cO < 4, "Invalid option");
        q.push(Question(_q, _O, _cO));
    }
    // 3 DO: choose option sO for question i
    function ansQ(uint i, uint8 sO) public {
        // RULE: question exists
        require(i < q.length, "Invalid question");
        // RULE: option is 0-3
        require(sO < 4, "Invalid option");
        // right if chosen option == correct option
        bool isc = sO == q[i].cO;
        if (isc) tc++; else tw++;
        // log it
        emit AnswerRes(q[i].Qtext, q[i].O[sO], isc);
    }
    // 4 SHOW (view): return one question
    function getQ(uint i) public view returns (string memory, string[4]
memory, uint8) {
        require(i < q.length, "Invalid question");
        return (q[i].Qtext, q[i].O, q[i].cO);
    }
    // 4 SHOW (view): return (correct, wrong)
    function getS() public view returns (uint cor, uint wro) { return (tc,
tw); }
}
```

Part 5 – Practical 7: Inheritance

Practical 7 – 1. Single inheritance (Vehicle → Car)

Car gets everything Vehicle has, and adds its own.

Remember: `contract Car is Vehicle` and the child constructor passes values up: Vehicle(typ, num).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

// PARENT contract: the common part
contract Vehicle {
    // 1 DATA: parent's fields
    string vehTyp;
    string vehNum;
    // 2 SETUP: fill the parent's fields
    constructor(string memory typ, string memory num) { vehTyp = typ; vehNum
= num; }
    // 4 SHOW (view)
    function display() public view returns (string memory) { return vehTyp; }
}

// CHILD: `is` = inherits everything from Vehicle
contract Car is Vehicle {
    // 1 DATA: child's own fields
    string color;
    string brand;
    // 2 SETUP: takes the parent's values + its own, and passes the parent's
up with Vehicle(typ, num)
    constructor(string memory typ, string memory num, string memory col,
string memory bnd) Vehicle(typ, num) {
        color = col;
        brand = bnd;
    }
    // 4 SHOW (view): child's own function
    function displayCar() public view returns (string memory) { return brand;
}
}
```

Practical 7 – 2. Multiple inheritance (StudentData + Marks → Grade)

Grade inherits from two contracts at once and combines them.

Remember: `is A, B` and the constructor lists both: A(...) B(...).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

// PARENT 1: name and roll number
contract StudentData {
```

```
    string name;
    string rollNo;
    constructor(string memory _name, string memory _rollNo) { name = _name;
rollNo = _rollNo; }
    function displayStudent() public view returns (string memory) { return
name; }
    function displayRollNo() public view returns (string memory) { return
rollNo; }
}

// PARENT 2: marks
contract Marks {
    uint public marks;
    constructor(uint m) { marks = m; }
    function displayMarks() public view returns (uint) { return marks; }
}

// CHILD: `is X, Y` = inherits from both
contract Grade is StudentData, Marks {
    // 2 SETUP: passes values up to BOTH parents
    constructor(string memory _name, string memory _rollNo, uint m)
StudentData(_name, _rollNo) Marks(m) {}

    // 4 SHOW (view): marks -> grade letter (marks comes from parent 2)
    function calculateGrade() public view returns (string memory) {
        if (marks >= 80) return "O";
        if (marks >= 70) return "A";
        if (marks >= 60) return "B";
        if (marks >= 50) return "C";
        return "F";
    }
    // 4 SHOW (view): joins parent 1's name and roll number with the grade
using string.concat
    function displayGrade() public view returns (string memory) {
        return string.concat("Name: ", displayStudent(), " Roll No: ",
displayRollNo(), " Grade: ", calculateGrade());
    }
}
```

Part 6 – Practical 8: Ether (payable)

Practical 8 – Q1: Deposit and show Ether

Anyone sends Ether in; the total is stored.

Remember: payable = can receive Ether. msg.value = how much was sent.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.1;

contract payex {
    // 1 DATA: total Ether received (in wei)
    uint public b;
    // 3 DO: `payable` lets the function receive Ether; msg.value is the
amount sent
    function deposit() public payable { b += msg.value; }
}
```

Practical 8 – Q2: Deposit and transfer Ether

Deposit Ether in, then send Ether on to another address.

Remember: Sending Ether to someone: addr.transfer(amount). The address must be `payable`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.1;

contract payex {
    // 1 DATA: total received
    uint public b;
    // 3 DO: receive Ether
    function deposit() public payable { b += msg.value; }
    // 3 DO: forward Ether to address r
    function transferE(address payable r) public payable {
        // RULE: the contract must hold enough (address(this).balance = the
contract's own Ether)
        require(address(this).balance >= msg.value, "Insufficient balance");
        // send the Ether to r
        r.transfer(msg.value);
    }
}
```

Practical 8 – Q2 (bank version)

Each address has its own balance: deposit, transfer to a friend, show, withdraw.

Remember: Bank = mapping(address => uint) + the same RULE before every subtract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.1;

contract SimpleBank {
    // 1 DATA: balance per wallet
    mapping(address => uint) public balance;
    // 3 DO: payable -> add msg.value to the sender's balance
    function deposit() public payable { balance[msg.sender] += msg.value; }
    // 3 DO: move money inside the bank (no real Ether leaves)
    function transfer(address to, uint amt) public {
        // RULE: enough money?
        require(balance[msg.sender] >= amt, "Insufficient balance");
        // subtract from sender
        balance[msg.sender] -= amt;
        // add to receiver
        balance[to] += amt;
    }
    // 4 SHOW (view): caller's balance
    function showBalance() public view returns (uint) { return
balance[msg.sender]; }
    // 3 DO: take real Ether out
    function withdraw(uint amt) public {
        require(balance[msg.sender] >= amt, "Insufficient balance");
        balance[msg.sender] -= amt;
        // send Ether to the caller (after reducing the balance)
        payable(msg.sender).transfer(amt);
    }
}
```

Practical 8 – Q3: Student fee payment

Students pay fees; the contract tracks each student's total and the overall total.

Remember: payable + require(msg.value > 0) + two totals.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentFeePayment {
    // 1 DATA: who deployed the contract
    address public owner = msg.sender;
    // 1 DATA: total of all fees
    uint256 public totalFeesCollected;
    // 1 DATA: fees per student (private = no free getter)
    mapping(address => uint256) private feesPaid;

    // 3 DO: pay a fee (payable)
    function payFee() public payable {
        // RULE: must send something
        require(msg.value > 0, "Fee must be greater than 0");
        // add to this student's total
        feesPaid[msg.sender] += msg.value;
        // add to the overall total
        totalFeesCollected += msg.value;
    }
    // 4 SHOW (view): my fees
    function viewMyFee() public view returns (uint256) { return
feesPaid[msg.sender]; }
    // 4 SHOW (view): all fees
    function viewTotalFeesCollected() public view returns (uint256) { return
totalFeesCollected; }
    // 4 SHOW (view): Ether the contract holds
    function getContractBalance() public view returns (uint256) { return
address(this).balance; }
}
```

Part 7 – Practical 9: Student Scholarship

Practical 9 – Student Scholarship

Owner registers students (eligible if marks > 75); eligible students claim a scholarship once.

Remember: Two contracts: `student` stores records (owner only), `scholarship is student` pays out. Every claim = a stack of requires.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

// PARENT: student records
contract student {
    // 1 DATA (a): one student's record
    struct Details { uint id; string name; string course; uint marks; bool
eligible; }
    // 1 DATA (b): id -> record
    mapping(uint => Details) public students;
    // 1 DATA (c): id exists?
    mapping(uint => bool) public isRegistered;
    // 1 DATA (d): owner = whoever deployed (msg.sender)
    address public owner = msg.sender;
    // log line
    event StudentAdded(uint id, string name);

    // a reusable RULE: check the caller is the owner, then `_;` = run the
function
    modifier onlyOwner() {
        require(msg.sender == owner, "This Action can only be performed by
the Owner!");
        _;
    }
    // 3 DO: only the owner can call (onlyOwner)
    function addStudent(uint id, string memory name, string memory course,
uint marks) public onlyOwner {
        // RULE: id must be new
        require(!isRegistered[id], "Student ID already exists!");
        // save the record; eligible is true only if marks > 75
        students[id] = Details(id, name, course, marks, marks > 75);   //
eligible only if marks > 75
        // mark as registered
        isRegistered[id] = true;
        // log it
        emit StudentAdded(id, name);
    }
    // 4 SHOW (view): read one record
    function getStudent(uint id) public view returns (uint, string memory,
string memory, uint, bool) {
        require(isRegistered[id], "Student Record not Found!");
        Details memory s = students[id];
        return (s.id, s.name, s.course, s.marks, s.eligible);
```

```
    }
}

// CHILD: inherits everything above
contract scholarship is student {
    // 1 DATA: fixed amount to pay
    uint public scholarshipAmount = 4;           // wei, fixed amount per
student
    // 1 DATA: already claimed?
    mapping(uint => bool) public hasClaimed;
    event FundsDeposited(uint amount);
    event ScholarshipClaimed(uint id, address to);

    // 3 DO: owner puts Ether into the contract (payable)
    function depositFunds() public payable onlyOwner { emit
FundsDeposited(msg.value); }

    // 3 DO: the claim. RULES in order:
    function claimScholarship(uint id) public {
        // registered
        require(isRegistered[id], "Student Record not found!");
        // eligible
        require(students[id].eligible, "Student Not Eligible!");
        // not claimed before
        require(!hasClaimed[id], "Scholarship Already Claimed!");
        // contract has enough money
        require(address(this).balance >= scholarshipAmount, "Insufficient
Funds");
        // mark claimed BEFORE paying (safe order)
        hasClaimed[id] = true;
        // pay the claimer
        payable(msg.sender).transfer(scholarshipAmount);
        emit ScholarshipClaimed(id, msg.sender);
    }
    // 4 SHOW (view): Ether in the contract
    function getTotalFunds() public view returns (uint) { return
address(this).balance; }
}
```

Part 8 – Practical 15: Inheritance and payments

Practical 15 – Q1: Online store

`product` stores the item; `onlineStore` adds buying.

Remember: Parent holds the data, child adds the payable function with two requires (stock + exact payment).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// PARENT: product details
contract product {
    // 1 DATA: id, name, price, stock
    uint productId;
    string productName;
    uint price;
    uint stock;

    // 2 SETUP: set the fields
    constructor(uint _id, string memory _name, uint _price, uint _stock) {
        productId = _id; productName = _name; price = _price; stock = _stock;
    }
    // 4 SHOW (view): return everything
    function displayProduct() public view returns (uint, string memory, uint,
uint) {
        return (productId, productName, price, stock);
    }
}

// CHILD: inherits the product
contract onlineStore is product {
    // 2 SETUP: passes the values to the parent: product(...)
    constructor(uint _id, string memory _name, uint _price, uint _stock)
product(_id, _name, _price, _stock) {}

    // 3 DO: buy q items (payable)
    function purchase(uint q) public payable {
        // RULE 1: enough stock
        require(stock >= q, "Product out of stock");
        // RULE 2: exact payment
        require(msg.value == q * price, "Wrong Payment");
```

```
        // reduce the stock
        stock -= q;
    }
}
```

Practical 15 – Q2: Vehicle rental

`Vehicle` holds details; `CarRental` rents it out for days and charges a penalty if returned late.

Remember: Rent: payable + require available + require enough money. Return: compare block.timestamp with the due time.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// PARENT: registration, brand, rent per day
contract Vehicle {
    string public vehicleRegistration;
    string public vehicleBrand;
    uint public vehicleRent;          // rent per day (wei)

    // 2 SETUP: set the fields
    constructor(string memory _reg, string memory _brand, uint _rent) {
        vehicleRegistration = _reg; vehicleBrand = _brand; vehicleRent =
_rent;
    }
}

// CHILD: adds renting
contract CarRental is Vehicle {
    // 1 DATA: who rents it, is it free, when is it due
    address public renter;
    bool public available = true;
    uint public dueTime;

    // logs
    event VehicleRented(address indexed renter, uint rentalDays, uint
amount);
    event VehicleReturned(address indexed renter, uint penalty);

    // 2 SETUP: pass details to the parent
    constructor(string memory _reg, string memory _brand, uint _rent)
```

```
Vehicle(_reg, _brand, _rent) {}

    // 3 DO: rent for _days (payable)
    function rentVehicle(uint _days) public payable {
        // RULE 1: must be free
        require(available, "Vehicle not available");
        // RULE 2: paid enough (rent x days)
        require(_days > 0 && msg.value >= vehicleRent * _days, "Insufficient
payment");
        // set three things at once: not free, who, due time (block.timestamp
= now)
        (available, renter, dueTime) = (false, msg.sender, block.timestamp +
_days * 1 minutes);   // 1 day = 1 minute for testing
        // log it
        emit VehicleRented(msg.sender, _days, msg.value);
    }

    // 4 SHOW (view): late? -> half the rent per late day, else 0
    function calculatePenalty() public view returns (uint) {
        if (available || block.timestamp <= dueTime) return 0;
        return ((block.timestamp - dueTime) / 1 minutes + 1) * vehicleRent /
2;   // half the rent per late day
    }

    // 3 DO: give it back
    function returnVehicle() public payable {
        // RULE 1: only the renter can return
        require(!available && msg.sender == renter, "Not the renter");
        // work out the penalty
        uint penalty = calculatePenalty();
        // RULE 2: penalty paid
        require(msg.value >= penalty, "Pay the penalty");
        // free again
        available = true;
        // log it
        emit VehicleReturned(msg.sender, penalty);
    }
}
```

Practical 15 – Q3: Doctor appointments

`Doctor` holds the fee; `Hospital` books and cancels slots with a refund rule.

Remember: Booking = mapping(slot => Booking). Book: 3 requires. Cancel: full refund if 1+ day away, else half.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// PARENT: doctor details and fee
contract Doctor {
    string public name;
    string public specialization;
    uint public fee;
    // `immutable` = set once in the constructor, never changes
    address public immutable owner;

    // 2 SETUP: set the fields; the deployer is the owner
    constructor(string memory _name, string memory _spec, uint _fee) {
        name = _name; specialization = _spec; fee = _fee; owner = msg.sender;
    }
}

// CHILD: adds bookings
contract Hospital is Doctor {
    // 1 DATA (a): who booked and what they paid
    struct Booking { address patient; uint paid; }
    // 1 DATA (b): slot time -> booking
    mapping(uint => Booking) public bookings;      // slot (unix time) =>
booking

    // logs
    event AppointmentBooked(address indexed patient, uint slot, uint paid);
    event AppointmentCancelled(address indexed patient, uint slot, uint
refund);

    // 2 SETUP: pass details to the parent
    constructor(string memory _name, string memory _spec, uint _fee)
Doctor(_name, _spec, _fee) {}

    // 3 DO: book a slot (payable)
    function bookAppointment(uint slot) external payable {
        // RULE 1: slot is in the future
        require(slot > block.timestamp, "Slot in the past");
        // RULE 2: slot is free (empty address = nobody)
        require(bookings[slot].patient == address(0), "Slot taken");
```

```
        // RULE 3: exact fee
        require(msg.value == fee, "Wrong fee");
        // save the booking
        bookings[slot] = Booking(msg.sender, msg.value);
        emit AppointmentBooked(msg.sender, slot, msg.value);
    }

    // 3 DO: cancel and get a refund
    function cancelAppointment(uint slot) external {
        // copy the booking
        Booking memory b = bookings[slot];
        // RULE 1: it is your booking
        require(b.patient == msg.sender, "Not your booking");
        // RULE 2: not already passed
        require(slot > block.timestamp, "Slot already passed");
        // full refund if 1+ day away, else half
        uint refund = slot - block.timestamp >= 1 days ? b.paid : b.paid / 2;
// full refund if 1+ day away, else 50%
        // free the slot
        delete bookings[slot];
        // send the refund
        (bool ok, ) = msg.sender.call{value: refund}("");
        // stop if the sending failed
        require(ok, "Refund failed");
        emit AppointmentCancelled(msg.sender, slot, refund);
    }

    // 3 DO: owner changes the fee
    function updateFee(uint _fee) external {
        // RULE: only the owner
        require(msg.sender == owner, "Only owner");
        fee = _fee;
    }
}
```

Part 9 – dApps: how to write them

A dApp is a contract plus a thin layer around it. Write it in this order, **C-D-F**:

1. **Contract** – write it with the 4 steps above.
2. **Deploy** – `migrations/1_deploy.js`, always the same two lines (Truffle). For Hardhat it is `scripts/deploy.js`.

3. **Frontend** – `index.html` (boxes, a button, a place for the result) and `app.js` (connect, build the contract object, on click: read boxes → call the function → show the result).

One rule for app.js: a `view` or `pure` function is read with `.call()` (free, no pop-up). Any other function is sent with `.send({ from: account })` (MetaMask asks you to confirm).

Practical 10 – Addition dApp (Truffle + Ganache + MetaMask)

1. contracts/Addition.sol

A pure function. Nothing stored.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Addition {
    // 4 SHOW (pure): the front end will call this
    function add(uint a, uint b) public pure returns (uint) { return a + b; }
}
```

2. migrations/1_deploy.js

```
// get the compiled contract by its contract name (Addition)
const Addition = artifacts.require("Addition");
// deploy it to the blockchain (this is the whole file)
module.exports = deployer => deployer.deploy(Addition);
```

3. truffle-config.js (add inside module.exports)

```
// tell Truffle where Ganache is: host 127.0.0.1, port 7545 (the port shown
in Ganache)
networks: {
  development: { host: "127.0.0.1", port: 7545, network_id: "*" },
},
// compiler version = the same as the pragma in your contract
compilers: {
  solc: { version: "0.8.21", settings: { optimizer: { enabled: false, runs:
200 }, evmVersion: "byzantium" } },
},
```

4. frontend/index.html

Shape to remember: input boxes with ids, a button that calls a function, a place for the result, then the two script tags (web3 first, your app.js second).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Addition App</title>
</head>
<body>
  <input type="number" id="n1" placeholder="enter first number"/><br/>
  <input type="number" id="n2" placeholder="enter second number"/><br/>
  <button onclick="addition()">addition</button>
  <h1>addition is : <span id="result"></span></h1>
  <script src="https://cdn.jsdelivr.net/npm/web3@4.16.0/dist/web3.min.js">
</script>
  <script src="app.js"></script>
</body>
</html>
```

5. frontend/app.js

```
// STEP 1: paste the address printed by `truffle migrate`
const contractAddress = "PASTE_CONTRACT_ADDRESS_HERE";
// STEP 2: ABI = the list of functions the contract has (name, inputs,
outputs). Copy from build/contracts/Addition.json
const contractABI = [{
  type: "function", name: "add", stateMutability: "pure",
  inputs: [{ internalType: "uint256", name: "a", type: "uint256" }, {
internalType: "uint256", name: "b", type: "uint256" }],
  outputs: [{ internalType: "uint256", name: "", type: "uint256" }],
}];

// STEP 3: runs when the button is clicked
async function addition() {
  try {
    // connect MetaMask
    await window.ethereum.request({ method: "eth_requestAccounts" });   //
connect MetaMask
    // build the contract object from the ABI + address
    const contract = new (new
```

```
Web3(window.ethereum)).eth.Contract(contractABI, contractAddress);
    // read the two input boxes
    const n1 = document.getElementById("n1").value, n2 =
document.getElementById("n2").value;
    if (n1 === "" || n2 === "") return alert("Please enter both numbers.");
    // call the function and show the answer. `.call()` = read-only, free
(add is pure)
    document.getElementById("result").innerText = await
contract.methods.add(n1, n2).call();
  // show any error
  } catch (error) {
    alert("Error: " + error.message);
  }
}
```

Practical 11 – Bank dApp

1. contracts/bank.sol

Same as the Practical 8 bank.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Bank {
    // 1 DATA: balance per wallet
    mapping(address => uint) public balance;

    // 3 DO: payable -> add msg.value to the sender
    function deposit() public payable { balance[msg.sender] += msg.value; }

    // 3 DO: take Ether out
    function withdraw(uint amount) public {
        // RULE: enough money?
        require(balance[msg.sender] >= amount, "Insufficient Balance");
        // subtract first
        balance[msg.sender] -= amount;
        // then send real Ether
        payable(msg.sender).transfer(amount);
    }
    // 3 DO: move balance to another wallet
    function transfer(address receiver, uint amount) public {
        require(balance[msg.sender] >= amount, "Insufficient Balance");
```

```
        balance[msg.sender] -= amount;
        // add to receiver
        balance[receiver] += amount;
    }
    // 4 SHOW (view): my balance
    function showBalance() public view returns (uint) { return
balance[msg.sender]; }
}
```

2. migrations/1_deploy.js

```
// get the contract by name, then deploy it
const Bank = artifacts.require("bank");
module.exports = deployer => deployer.deploy(Bank);
```

3. frontend/index.html

One section per action: a box for the amount and a button. Every button has an id that app.js uses.

```
<!DOCTYPE html>
<html>
<head><title>Simple Bank DApp</title></head>
<body>
  <h1>Simple Bank</h1>
  <button id="connectButton">Connect MetaMask</button>
  <p>Account: <span id="account">Not connected</span></p>

  <h2>Balance</h2>
  <p><span id="balance">0</span> ETH</p>
  <button id="refreshButton">Refresh Balance</button>
  <hr>
  <h2>Deposit</h2>
  <input id="depositAmount" type="number" placeholder="ETH amount">
  <button id="depositButton">Deposit</button>
  <hr>
  <h2>Withdraw</h2>
  <input id="withdrawAmount" type="number" placeholder="ETH amount">
  <button id="withdrawButton">Withdraw</button>
  <hr>
  <h2>Transfer</h2>
  <input id="receiver" type="text" placeholder="Receiver address">
```

```
  <input id="transferAmount" type="number" placeholder="ETH amount">
  <button id="transferButton">Transfer</button>
  <p id="status"></p>

  <script src="https://cdn.jsdelivr.net/npm/web3@4.16.0/dist/web3.min.js">
</script>
  <script src="app.js"></script>
</body>
</html>
```

4. frontend/app.js

```
// paste the deployed address
const BANK_ADDRESS = "PASTE_DEPLOYED_BANK_ADDRESS";
// ABI: only the 4 functions this page uses. stateMutability tells you the
type: payable / nonpayable / view
const BANK_ABI = [   // only the four functions the page uses (same as the
ABI in build/contracts/Bank.json)
  { type: "function", name: "deposit", stateMutability: "payable", inputs:
[], outputs: [] },
  { type: "function", name: "withdraw", stateMutability: "nonpayable",
inputs: [{ name: "amount", type: "uint256" }], outputs: [] },
  { type: "function", name: "transfer", stateMutability: "nonpayable",
inputs: [{ name: "receiver", type: "address" }, { name: "amount", type:
"uint256" }], outputs: [] },
  { type: "function", name: "showBalance", stateMutability: "view", inputs:
[], outputs: [{ name: "", type: "uint256" }] },
];

// global variables: web3, the contract and the connected account
let web3, bank, user;
// tiny helper: $("id") instead of document.getElementById
const $ = id => document.getElementById(id);
// reads an ETH box and converts it to wei
const wei = id => web3.utils.toWei($(id).value, "ether");        // ETH box -
> wei

// 1 connect MetaMask, 2 create the contract object, 3 show the account and
balance
async function connect() {                                       // connect
MetaMask
  [user] = await ethereum.request({ method: "eth_requestAccounts" });
  web3 = new Web3(ethereum);
```

```
  bank = new web3.eth.Contract(BANK_ABI, BANK_ADDRESS);
  $("account").innerText = user;
  getBalance();
}
// READ: showBalance is `view` -> `.call({ from: user })` (from is needed
because the contract uses msg.sender)
async function getBalance() {                                    // show bank
balance
  $("balance").innerText = web3.utils.fromWei(await
bank.methods.showBalance().call({ from: user }), "ether");
}
// WRITE: every function that changes data uses `.send({ from: user })`
(MetaMask asks to confirm); then refresh the balance
async function send(txn, status, value) {                       // send
transaction, refresh, show status
  await txn.send({ from: user, value });
  await getBalance();
  $("status").innerText = status;
}

// Wire each button to its action. Deposit also sends Ether (the third value)
$("connectButton").onclick = connect;
$("refreshButton").onclick = getBalance;
$("depositButton").onclick = () => send(bank.methods.deposit(), "Deposit
successful!", wei("depositAmount"));
$("withdrawButton").onclick = () =>
send(bank.methods.withdraw(wei("withdrawAmount")), "Withdrawal successful!");
$("transferButton").onclick = () =>
send(bank.methods.transfer($("receiver").value, wei("transferAmount")),
"Transfer successful!");
```

Practical 12 – Prime dApp

1. contracts/Prime.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Prime {
    // 4 SHOW (pure): number in, text out
    function isPrime(uint256 num) public pure returns (string memory) {
        // reject 0 and 1
        if (num <= 1) return "Please enter a value above one!";
        // try divisors up to the square root
        for (uint256 i = 2; i * i <= num; i++)
            // found a divisor -> not prime
            if (num % i == 0) return "The entered number is not a prime
number.";
        // no divisor found -> prime
        return "The entered number is a prime number.";
    }
}
```

2. migrations/1_deploy.js

```
// get the contract by name, then deploy it
const Prime = artifacts.require("Prime");
module.exports = deployer => deployer.deploy(Prime);
```

3. frontend/index.html

Same shape as Practical 10 with one input.

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Prime Checker</title></head>
<body>
  <input type="number" id="n" placeholder="Enter a number"/>
  <button onclick="checkPrime()">Check Prime</button>
  <h1><span id="result"></span></h1>
  <script src="https://cdn.jsdelivr.net/npm/web3@4.16.0/dist/web3.min.js">
</script>
  <script src="app.js"></script>
</body>
</html>
```

4. frontend/app.js

```
// paste the deployed address
const contractAddress = "PASTE_CONTRACT_ADDRESS_HERE";
// ABI of isPrime (pure, one number in, a string out)
const contractABI = [{   // same as the isPrime entry in
build/contracts/Prime.json
  type: "function", name: "isPrime", stateMutability: "pure",
  inputs: [{ internalType: "uint256", name: "num", type: "uint256" }],
  outputs: [{ internalType: "string", name: "", type: "string" }],
}];

// same 4 steps as Practical 10: connect MetaMask, build the contract, read
the input box, call and show the result
async function checkPrime() {
  if (!window.ethereum) return alert("Please install MetaMask!");
  try {
    await ethereum.request({ method: "eth_requestAccounts" });
    const contract = new (new Web3(ethereum)).eth.Contract(contractABI,
contractAddress);
    const n = document.getElementById("n").value;
    if (n === "") return alert("Please enter a number.");
    // `.call()` because the function is pure
    document.getElementById("result").innerText = await
contract.methods.isPrime(n).call();
  } catch (e) {
    alert("Error: " + e.message);
  }
}
```

Practical 13 A – Hardhat addition dApp

1. contracts/addition.sol

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Addition {
    // 4 SHOW (pure)
    function add(uint a, uint b) public pure returns (uint) { return a + b; }
}
```

2. hardhat.config.ts

```
import { defineConfig } from "hardhat/config";
import hardhatToolboxMochaEthers from "@nomicfoundation/hardhat-toolbox-
mocha-ethers";

export default defineConfig({
  // the plugin gives Mocha tests + ethers support
  plugins: [hardhatToolboxMochaEthers],
  // compiler version
  solidity: "0.8.28",
});
```

3. scripts/deploy.js

```
import { network } from "hardhat";

// connect to the network chosen on the command line (--network localhost)
const { ethers } = await network.create();

// get the contract factory = the 'builder' of the contract
const Addition = await ethers.getContractFactory("Addition");
// deploy it
const addition = await Addition.deploy();
// wait until it is mined
await addition.waitForDeployment();

// print the address; paste it into app.js
console.log("Addition deployed to:", await addition.getAddress());
```

4. frontend/index.html

Same shape as Practical 10; the only difference is the ethers script tag instead of web3.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Hardhat Addition DApp</title></head>
<body>
  <h2>Addition DApp</h2>
  <input id="a" type="number" placeholder="a"> +
  <input id="b" type="number" placeholder="b">
  <button id="add">Add</button>
  <p>Result: <span id="out"></span></p>
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/ethers/6.13.4/ethers.umd.min.js">
</script>
  <script src="app.js"></script>
</body>
</html>
```

5. frontend/app.js

Hardhat version: ethers (not web3) with a short ABI. No MetaMask for a pure function.

```
// paste the address printed by deploy.js
const CONTRACT_ADDRESS = "PASTE_DEPLOYED_ADDRESS";
// where the local Hardhat node listens
const RPC_URL = "http://127.0.0.1:8545";
// short 'human-readable' ABI: just the function signature
const ABI = ["function add(uint256 a, uint256 b) pure returns (uint256)"];

// provider (the node) + ABI + address = a contract you can read from; no
MetaMask needed for pure/view calls
const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, new
ethers.JsonRpcProvider(RPC_URL));
const $ = id => document.getElementById(id);

// on click: read the boxes, call add, show the result (BigInt -> text with
toString)
$("add").onclick = async () => {
  try {
    const r = await contract.add($("a").value, $("b").value);
    $("out").innerText = r.toString();
```

```
  } catch (e) {
    $("out").innerText = "Error: " + e.message;
  }
};
```

Practical 13 B – Voting dApp (Truffle)

1. contracts/voting.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.21;

contract Voting {
    // 1 DATA (a): one candidate record
    struct Candidate { uint256 id; string name; uint256 voteCount; }

    // 1 DATA (b): owner
    address public owner;
    // 1 DATA (c): how many candidates (also used as the next id)
    uint256 public candidateCount;
    // 1 DATA (d): id -> candidate
    mapping(uint256 => Candidate) public candidates;
    // 1 DATA (e): has this wallet voted?
    mapping(address => bool) public hasVoted;

    // logs for the front end
    event CandidateAdded(uint256 id, string name);
    event VoteCast(address voter, uint256 candidateId);

    // 2 SETUP: the deployer becomes the owner
    constructor() { owner = msg.sender; }

    // 3 DO: owner adds a candidate
    function addCandidate(string memory _name) public {
        // RULE: only the owner
        require(msg.sender == owner, "Only owner");
        // RULE: name not empty
        require(bytes(_name).length > 0, "Name required");
        // next id
        candidateCount++;
        // store: id, name, 0 votes
        candidates[candidateCount] = Candidate(candidateCount, _name, 0);
```

```
        // log it
        emit CandidateAdded(candidateCount, _name);
    }

    // 3 DO: vote for candidate _id
    function vote(uint256 _id) public {
        // RULE: not voted yet
        require(!hasVoted[msg.sender], "Already voted");
        // RULE: the candidate exists
        require(_id > 0 && _id <= candidateCount, "Invalid candidate");
        // remember the vote
        hasVoted[msg.sender] = true;
        // add the vote
        candidates[_id].voteCount++;
        emit VoteCast(msg.sender, _id);
    }

    // 4 SHOW (view): id, name, votes
    function getCandidate(uint256 _id) public view returns (uint256, string
memory, uint256) {
        Candidate memory c = candidates[_id];
        return (c.id, c.name, c.voteCount);
    }
}
```

2. migrations/1_deploy.js

```
// get the contract by name, then deploy it
const Voting = artifacts.require("Voting");
module.exports = deployer => deployer.deploy(Voting);
```

3. frontend/index.html

A list that app.js fills with candidates, a box and button to add one, and a status line.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Voting DApp</title></head>
<body>
  <h2>Voting DApp</h2>
  <ul id="list"></ul>
  <input id="name" placeholder="Candidate name">
  <button id="addBtn">Add Candidate</button>
  <p id="status"></p>
  <script src="https://cdn.jsdelivr.net/npm/web3@4.16.0/dist/web3.min.js">
</script>
  <script src="app.js"></script>
</body>
</html>
```

4. frontend/app.js

```
// paste the deployed address
const ADDRESS = "PASTE_DEPLOYED_ADDRESS";
// only the functions the page uses
const ABI = [
  { type: "function", name: "candidateCount", stateMutability: "view",
inputs: [], outputs: [{ type: "uint256" }] },
  { type: "function", name: "addCandidate", stateMutability: "nonpayable",
inputs: [{ name: "_name", type: "string" }], outputs: [] },
  { type: "function", name: "vote", stateMutability: "nonpayable", inputs: [{
name: "_id", type: "uint256" }], outputs: [] },
  { type: "function", name: "getCandidate", stateMutability: "view", inputs:
[{ name: "_id", type: "uint256" }],
    outputs: [{ type: "uint256" }, { type: "string" }, { type: "uint256" }]
},
];

let contract, user;
const $ = id => document.getElementById(id);

// READ: get candidateCount, loop 1..count, call getCandidate(i) and build a
list item with a Vote button
async function load() {                                          // list
candidates with vote counts
  const n = Number(await contract.methods.candidateCount().call());
  $("list").innerHTML = "";
```

```
  for (let i = 1; i <= n; i++) {
    const c = await contract.methods.getCandidate(i).call();
    const li = document.createElement("li");
    li.innerHTML = `${c[1]} – ${c[2]} votes `;
    const b = document.createElement("button");
    b.textContent = "Vote";
    b.onclick = () => run(contract.methods.vote(i), "Vote cast!");
    li.appendChild(b);
    $("list").appendChild(li);
  }
}
// WRITE helper: send the transaction, show a message, reload the list
async function run(txn, ok) {                                    // send tx,
report, reload
  try { await txn.send({ from: user }); $("status").innerText = ok; await
load(); }
  catch (e) { $("status").innerText = "Error: " + e.message; }
}
// Add-candidate button
$("addBtn").onclick = () =>
run(contract.methods.addCandidate($("name").value), "Candidate added!");

// at page load: connect MetaMask, build the contract, show the list
(async () => {
  [user] = await ethereum.request({ method: "eth_requestAccounts" });
  contract = new (new Web3(ethereum)).eth.Contract(ABI, ADDRESS);
  load();
})();
```